



Biomedical Signal Processing Lab(LAB Master DSAI 2025) - Professor Olivier Meste

Prabal Ghosh¹[0009–0004–3449–5811]

Universite Cote d'Azur, Sophia Antipolis, France
prabal5ghosh@gmail.com

1 Problem Statement

The matrix Rva and Ra (splitted in Rva1.mat, Rva2.mat, Rva3.mat and Ra1.mat, Ra2.mat, Ra3.mat) contain tensors of 75 patients with Atrial Fibrillation (AF) cardiac pathology. They consist in 15000 samples of 12 Leads-ECG records order as : I, II, III, aVR, aVL, aVF, V1, V2, V3, V4, V5, V6. The sampling frequency is 256 Hertz.

The ECG in matrix Rva is the raw signals (but band-pass filtered) including ventricular and auricular electrical activities (depolarizations and repolarizations). The ECG in matrix Ra is the modified records where the ventricular activities are removed (complex processing). Only atrial fibrillation signal remains.

All subjects experienced an Electrical Cardioversion to restore sinus rhythm (SR). After a blanking period of 6 months, some of them returned to AF (file indrecur.mat) and the rest remained SR (file indnonrecur.mat). Only 63 subjects are documented in indnonrecur and indrecur vectors (indnonrecur.mat, indrecur.mat)

```
load('C:\Users\praba\Documents\GitHub\UCA-M2-SEMESTER2\bio medical signal processing\lab
    assignments\Atrial_prabal_ghosh\Rva1.mat');
Xva1 = Rva1;

load('C:\Users\praba\Documents\GitHub\UCA-M2-SEMESTER2\bio medical signal processing\lab
    assignments\Atrial_prabal_ghosh\Rva2.mat');
Xva2 = Rva2;

load('C:\Users\praba\Documents\GitHub\UCA-M2-SEMESTER2\bio medical signal processing\lab
    assignments\Atrial_prabal_ghosh\Rva3.mat');
Xva3 = Rva3;

% Merge along the 3rd dimension (patient axis)
Xva_all = cat(3, Xva1, Xva2, Xva3);

fprintf('Total patients loaded: %d\n', size(Xva_all, 3));

%%
load('C:\Users\praba\Documents\GitHub\UCA-M2-SEMESTER2\bio medical signal processing\lab
    assignments\Atrial_prabal_ghosh\Ra1.mat');
Xa1 = Ra1;

load('C:\Users\praba\Documents\GitHub\UCA-M2-SEMESTER2\bio medical signal processing\lab
    assignments\Atrial_prabal_ghosh\Ra2.mat');
Xa2 = Ra2;

load('C:\Users\praba\Documents\GitHub\UCA-M2-SEMESTER2\bio medical signal processing\lab
    assignments\Atrial_prabal_ghosh\Ra3.mat');
Xa3 = Ra3;

Xa_all = cat(3, Xa1, Xa2, Xa3);
```

Listing 1.1: MATLAB Code for Loading and Merging Data

1.1 1. Visualisation of the 12 Leads-ECG (with Matlab)

```

lead = 1;           % Lead I
subject = 1;        % First subject

signal = squeeze(Xva_all(lead, :, subject)); % Get 1D signal for one lead and subject
fs = 256;           % Sampling frequency

%% STEP 1: Visualize 12-lead ECG for this subject
figure;
for i = 1:12
    subplot(6, 2, i);
    plot(squeeze(Rva1(i, :, subject)));
    title(['Lead ', num2str(i)]);
    xlabel('Samples');
    ylabel('Amplitude');
end
%sgtitle('12-Lead ECG for Subject 1');

```

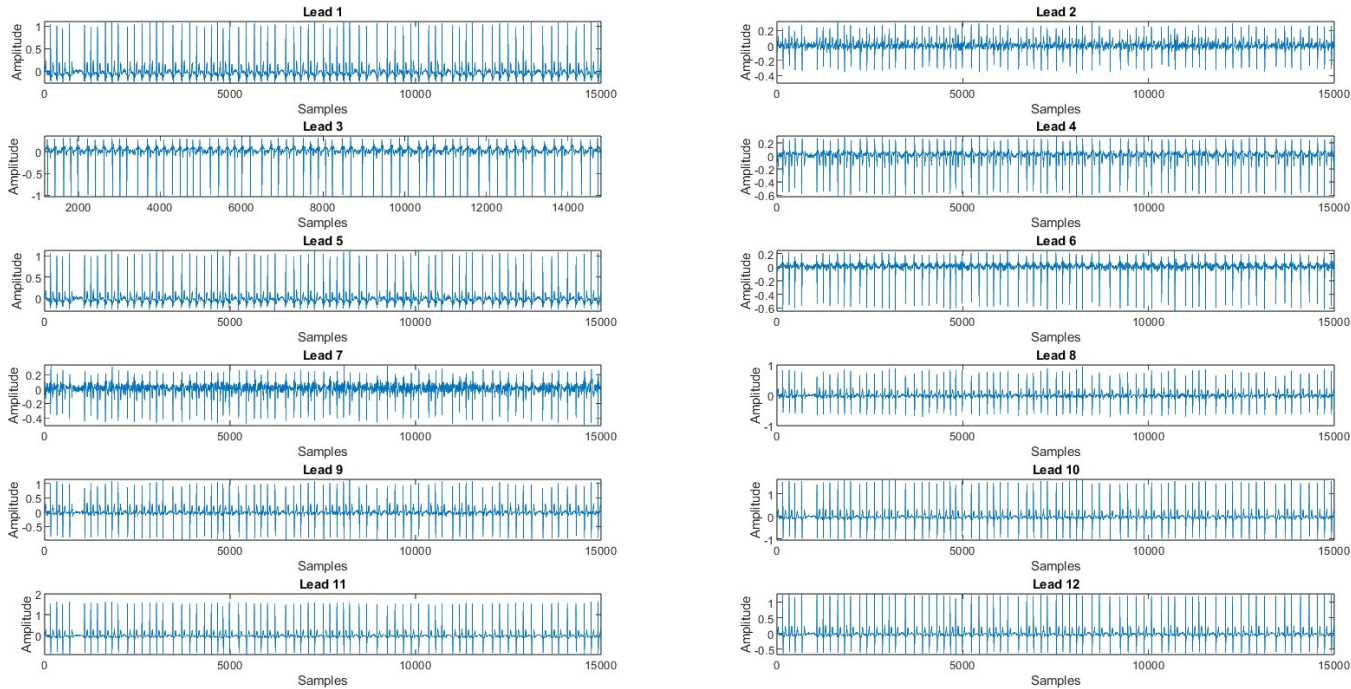


Fig. 1: 12-Lead ECG signal for Subject 1

1.2 2) Simple detection of R waves (with Matlab)

```

signal = squeeze(Xva_all(1, :, 1)); % Lead I, Subject 1
fs = 256;
threshold = max(signal) * 0.5;
min_dist = round(0.2 * fs);

```

```

LOCS = [];
PKS = [];
last_peak = -inf;

for i = 2:length(signal)-1
    if signal(i) > threshold && signal(i) > signal(i-1) && signal(i) > signal(i+1)
        if isempty(LOCS) || (i - last_peak) > min_dist
            LOCS = [LOCS i];
            PKS = [PKS signal(i)];
            last_peak = i;
        end
    end
end

figure
plot(signal)
hold on
plot(LOCS, PKS, 'ro')
title('R-peaks Detected - Lead I, Subject 1')
xlabel('Samples'); ylabel('Amplitude (mV)')

```

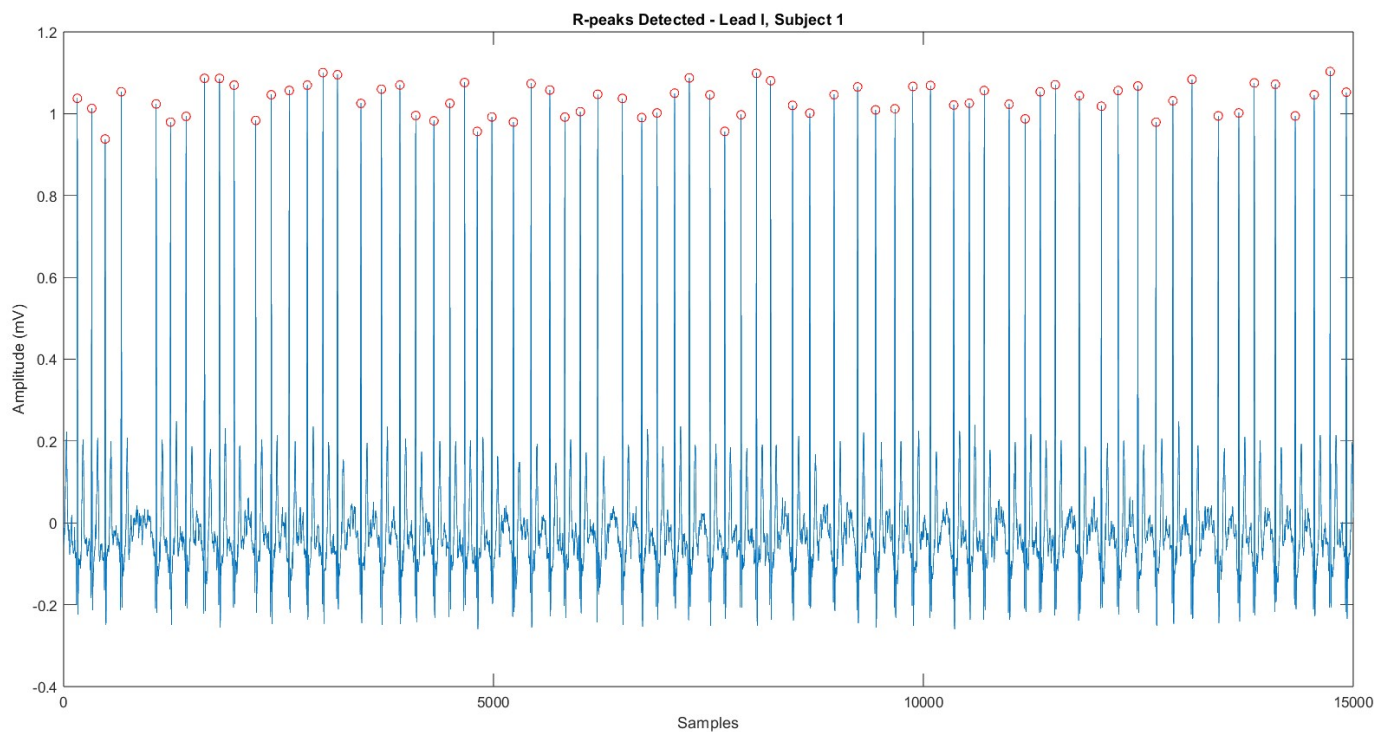


Fig. 2: R-peaks Detected - Lead I, Subject 1

1.3 3) QRST averaging (with Matlab)

```

%LOCS = R-peaks detected earlier
R = []; % This will hold all QRST segments
fs = 256;
before = 20;
after = min(diff(LOCS)) - 21; % conservative end point to avoid overlap

```

```

% Build the R matrix from Lead I, Subject 1
for k = 2:length(LOCS)-1
    segment = Xva_all(1, LOCS(k)-before : LOCS(k)+after, 1);
    R(k-1, :) = segment;
end

% Visualize all segments
figure;
plot(R'); % each row = one QRST
title('Aligned QRST Segments (Lead I)');

% Average QRST waveform
mean_QRST = mean(R);
figure;
plot(mean_QRST, 'r', 'LineWidth', 2);
title('Mean QRST Complex');

```

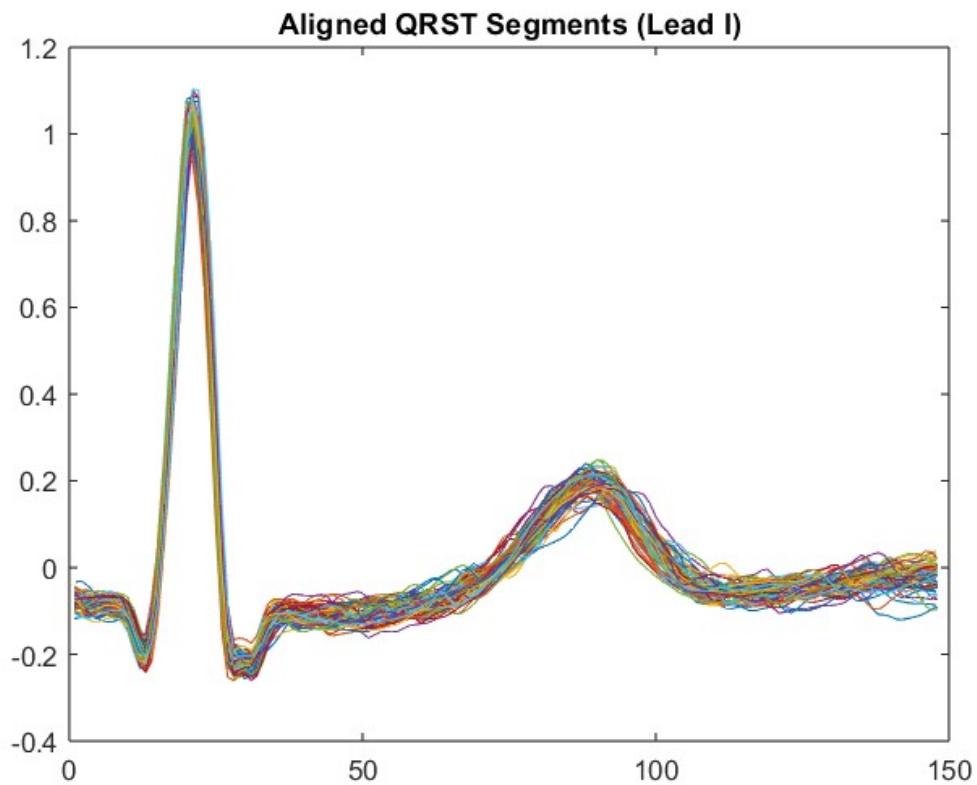


Fig. 3: Aligned QRST Segments (Lead I)

1.4 4) Ventricular activity subtraction : SVD-based (with Matlab)

```

fs = 256; % Sampling frequency
lead = 1; % First lead (Lead I)

% Extract QRST segments from Xva (raw ECG)
R = []; % This will hold all QRST segments
before = 20;
after = min(diff(LOCS)) - 21;

% Build the R matrix from Lead I
for k = 2:length(LOCS)-1

```

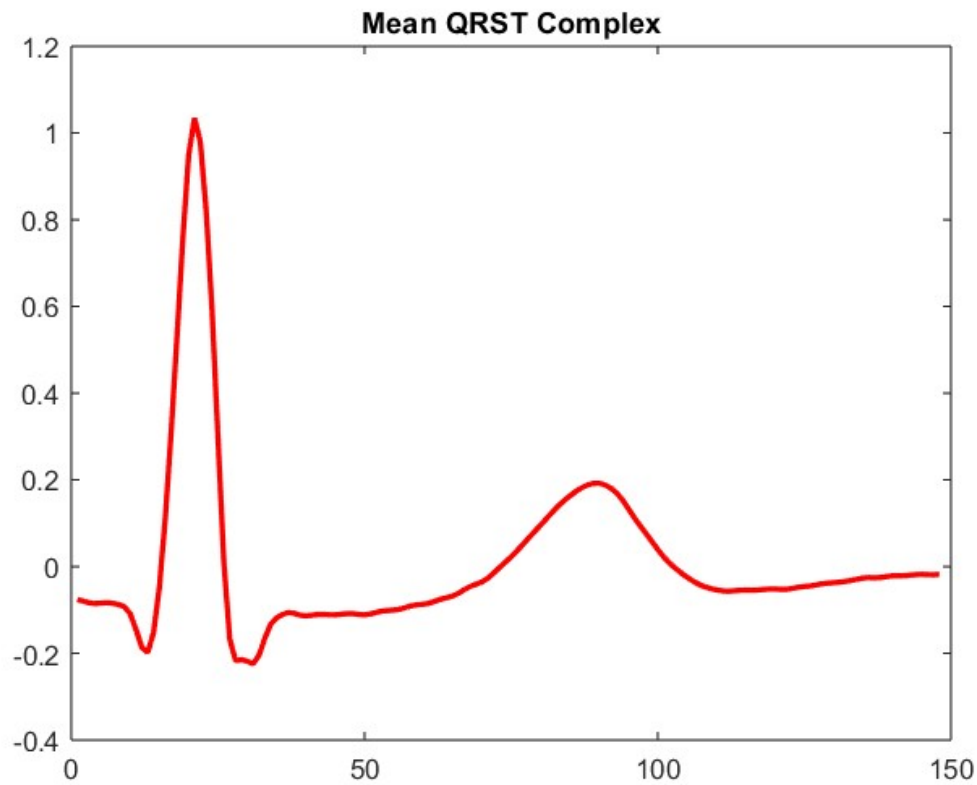


Fig. 4: Mean QRST Complex

```

    segment = Xva_all(lead, LOCS(k)-before : LOCS(k)+after, 1);
    R(k-1, :) = segment;
end

% Perform SVD on the QRST segment matrix
[U, S, V] = svd(R); % R is the matrix of all QRST segments

% Create a "ventricular subspace" from the first 2 components
M = [V(:, 1), V(:, 2)]; % QRS + T-wave basis

% Project the first beat onto the ventricular subspace
a = pinv(M) * R(1, :)' ; % Coefficients for reconstruction
reconstructed = M * a; % Reconstruct ventricular activity

% Subtract ventricular activity to get atrial residual
residual = R(1, :)' - reconstructed;

% Plot the result of ventricular activity subtraction
figure;
subplot(3, 1, 1);
plot(R(1, :)', 'b');
title('Original QRST Segment');
xlabel('Samples'); ylabel('Amplitude');

subplot(3, 1, 2);
plot(reconstructed, 'r');
title('Reconstructed Ventricular Activity (QRS+T)');
xlabel('Samples'); ylabel('Amplitude');

subplot(3, 1, 3);
plot(residual, 'k');
title('Residual Atrial Activity');

```

```

xlabel('Samples'); ylabel('Amplitude');

% Extract corresponding atrial activity segment from Xa
Ra = []; % This will hold atrial activity segments
for k = 2:length(LOCS)-1
    segment = Xa_all(lead, LOCS(k)-before : LOCS(k)+after, 1);
    Ra(k-1, :) = segment;
end

% Compare residual atrial activity with Xa
figure;
hold off;
plot(Ra(1, :)', 'g', 'LineWidth', 1.5); % Atrial activity from Xa
hold on;
plot(residual, 'k', 'LineWidth', 1.5); % Residual atrial activity from Xva
title('Comparison of Atrial Activity (Xa vs Residual)');
xlabel('Samples'); ylabel('Amplitude');
legend('Xa (Atrial Activity)', 'Residual (From Xva)');

```

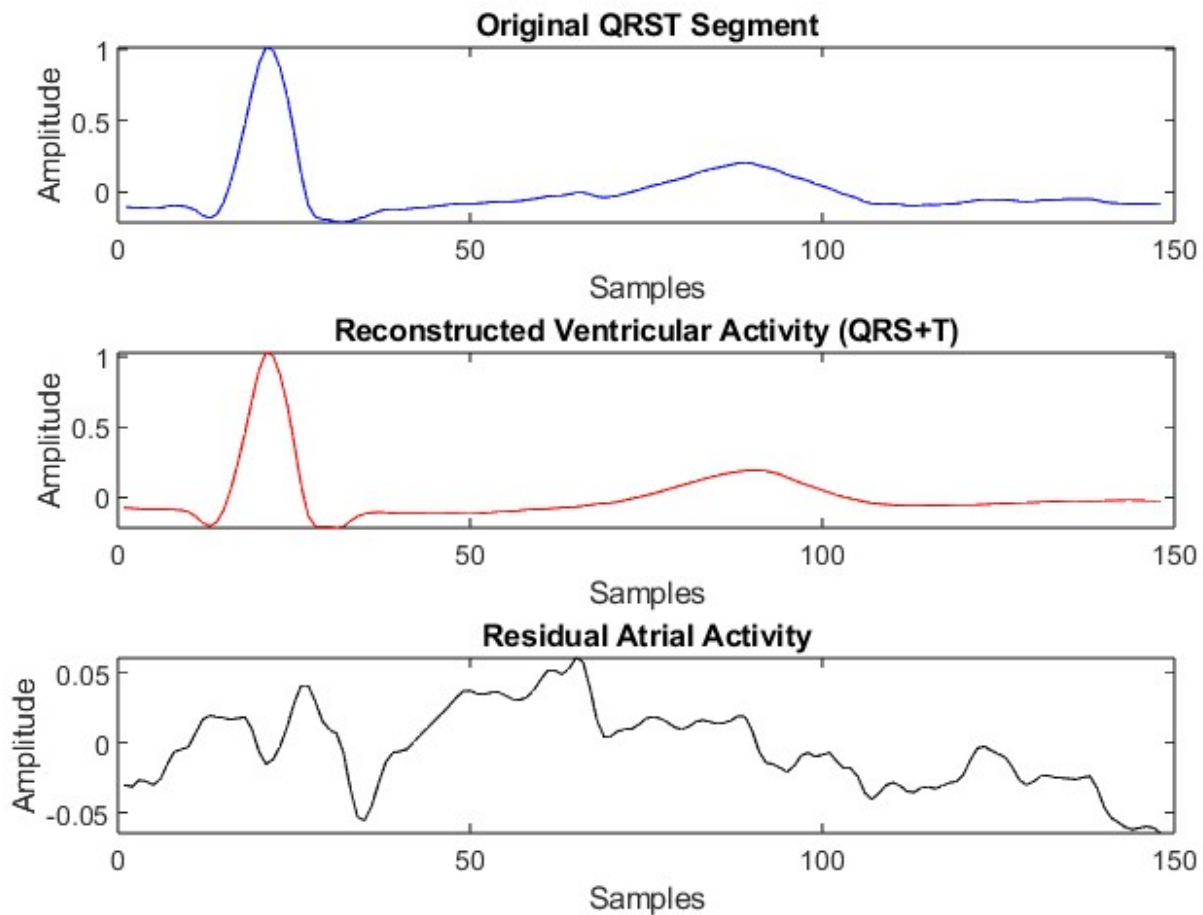


Fig. 5: plotting Original QRST Segment, Reconstructed Ventricular Activity (QRS+T), Residual Atrial Activity

- **Original QRST Segment:** This plot shows the raw QRST segment extracted from the ECG signal (Xva). It includes both ventricular activity (QRS complex and T-wave) and atrial activity.
- **Reconstructed Ventricular Activity (QRS + T-wave):** This plot shows the ventricular activity reconstructed using the first two singular vectors from the SVD. It captures the dominant components of the QRS complex and T-wave.

- **Residual Atrial Activity:** This plot shows the residual signal after subtracting the reconstructed ventricular activity from the original QRST segment. The residual primarily represents the atrial activity.

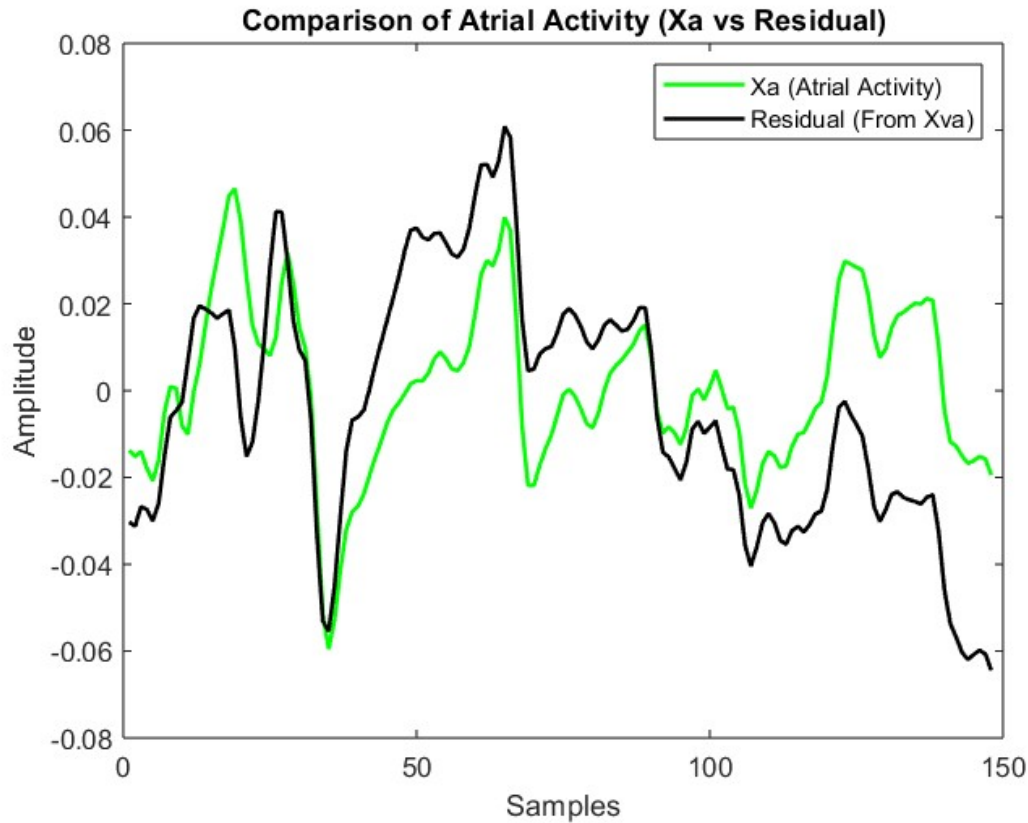


Fig. 6: Comparison of Atrial Activity (Xa vs Residual)

- **Atrial Activity from Xa:** This plot shows the atrial activity segment extracted from the atrial activity matrix (Xa). It serves as a reference for comparison.
- **Residual Atrial Activity from Xva:** This plot shows the residual atrial activity obtained after subtracting the ventricular activity from the raw ECG signal (Xva). It is compared with the atrial activity from Xa to validate the subtraction process.

1.5 5) Machine learning for AF recurrence classification (applied to Ra) (student choice for the software, Matlab ?,Tensorflow ?) à Use only V1 to V6 leads (Ra(7:12,:))

Model 1. For this classification, I used random forest from sklearn.

```
import numpy as np
import scipy.io as sio
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from scipy.signal import welch
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score,
    ConfusionMatrixDisplay
from scipy.stats import entropy
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

# === LOAD & PREP DATA ===
# Load Ra files
```



```

ra1 = sio.loadmat('C:\\Users\\praba\\Documents\\GitHub\\UCA-M2-SEMESTER2\\bio medical signal
processing\\lab assignments\\Atrial_prabal_ghosh\\Ra1.mat')['Ra1']
ra2 = sio.loadmat('C:\\Users\\praba\\Documents\\GitHub\\UCA-M2-SEMESTER2\\bio medical signal
processing\\lab assignments\\Atrial_prabal_ghosh\\Ra2.mat')['Ra2']
ra3 = sio.loadmat('C:\\Users\\praba\\Documents\\GitHub\\UCA-M2-SEMESTER2\\bio medical signal
processing\\lab assignments\\Atrial_prabal_ghosh\\Ra3.mat')['Ra3']

# Load Ra files
ra1 = sio.loadmat('C:\\Users\\praba\\Documents\\GitHub\\UCA-M2-SEMESTER2\\bio medical signal
processing\\lab assignments\\Atrial_prabal_ghosh\\Ra1.mat')['Ra1']
ra2 = sio.loadmat('C:\\Users\\praba\\Documents\\GitHub\\UCA-M2-SEMESTER2\\bio medical signal
processing\\lab assignments\\Atrial_prabal_ghosh\\Ra2.mat')['Ra2']
ra3 = sio.loadmat('C:\\Users\\praba\\Documents\\GitHub\\UCA-M2-SEMESTER2\\bio medical signal
processing\\lab assignments\\Atrial_prabal_ghosh\\Ra3.mat')['Ra3']

# Merge into one big 3D array      shape: (12, 15000, 75)
Xa_all = np.concatenate([ra1, ra2, ra3], axis=2)

# Extract only V1 V6 (indices 6 to 11)
Xa_v1_v6 = Xa_all[6:12, :, :] # shape: (6, 15000, 75)

# Rearrange to (75 patients, 15000 time steps, 6 leads)
X = np.transpose(Xa_v1_v6, (2, 1, 0)) # shape: (75, 15000, 6)

# === LABELS ===

# Load recurrence labels
indrecur = sio.loadmat('C:\\Users\\praba\\Documents\\GitHub\\UCA-M2-SEMESTER2\\bio medical
signal processing\\lab assignments\\Atrial_prabal_ghosh\\indrecur.mat')['indrecur'].flatten
() - 1
indnonrecur = sio.loadmat('C:\\Users\\praba\\Documents\\GitHub\\UCA-M2-SEMESTER2\\bio medical
signal processing\\lab assignments\\Atrial_prabal_ghosh\\indnonrecur.mat')['indnonrecur'].
flatten() - 1

# Create label vector: 0 = no recurrence, 1 = recurrence
y = np.zeros(75)
y[indrecur] = 1

# Feature extraction function
def extract_features(signal, fs=256):
    features = []
    for lead in range(signal.shape[1]):
        x = signal[:, lead]
        features.append(np.mean(x))
        features.append(np.std(x))
        features.append(np.max(x) - np.min(x))
        f, Pxx = welch(x, fs=fs)
        features.append(np.sum(Pxx)) # signal power
        features.append(np.argmax(Pxx)) # peak frequency index
        features.append(entropy(np.abs(Pxx + 1e-8))) # spectral entropy
    return np.array(features)

# Extract features for each patient
X_features = np.array([extract_features(x) for x in X]) # [75, features]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_features, y, test_size=0.2, stratify=y,
random_state=42)

```



```

# Train Random Forest
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

# Evaluate
y_pred = clf.predict(X_test)
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred, output_dict=True)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Non-Recurrence', 'Recurrence'],
            yticklabels=['Non-Recurrence', 'Recurrence'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.show()

print("accuracy_score", acc)
print("classification_report", report)

```

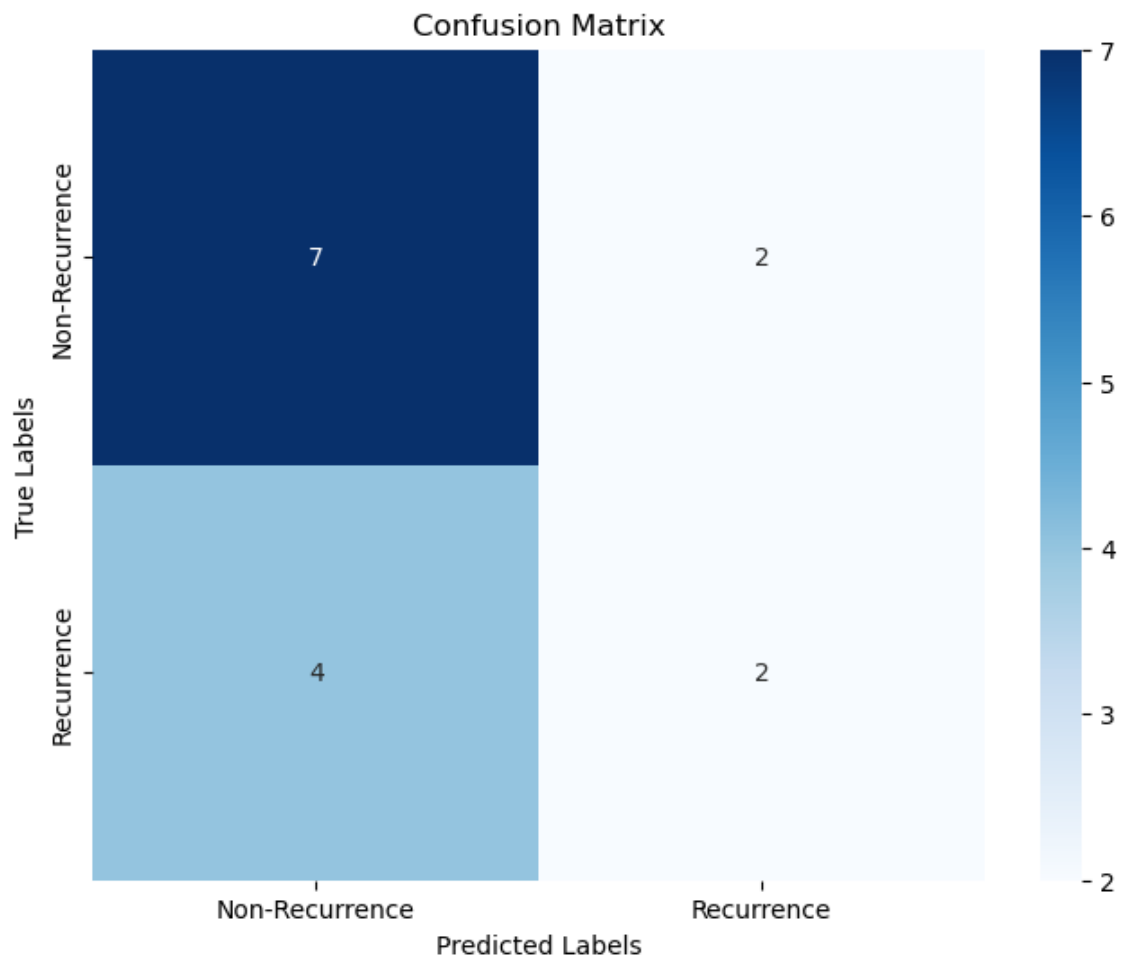


Fig. 7: Confusion Matrix

The accuracy and other classification report is given below.

```
accuracy_score 0.6
```

```

classification_report {'0.0': {'precision': 0.6363636363636364, 'recall': 0.7777777777777778, '
    f1-score': 0.7, 'support': 9.0}, '1.0': {'precision': 0.5, 'recall': 0.3333333333333333, 'f1
    -score': 0.4, 'support': 6.0}, 'accuracy': 0.6, 'macro avg': {'precision':
    0.5681818181818181, 'recall': 0.5555555555555556, 'f1-score': 0.55, 'support': 15.0}, '
    weighted avg': {'precision': 0.5818181818181818, 'recall': 0.6, 'f1-score': 0.58, 'support':
    15.0}}

```

From the confusion matrix, accuracy, and F1 score, it is clear that the classification model is not performing well. This suggests that the current approach, which uses a Random Forest classifier, is not suitable for this specific time-series data. Random Forest is a simple and powerful machine learning model that works well for many classification tasks, especially when the data is tabular and lacks temporal dependencies. However, it has significant limitations when applied to time-series data.

Time-series data inherently contains sequential patterns and temporal dependencies that Random Forest cannot capture. This is because Random Forest treats each feature independently and does not consider the order or relationship between time steps. As a result, it fails to leverage the temporal structure of the data, which is often critical for accurate classification in time-series tasks.

To address this limitation, more advanced models like Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), or Bidirectional LSTMs (BiLSTMs) should be considered. These models are specifically designed to handle sequential data and can learn temporal dependencies effectively. By using these models, the classification performance can be significantly improved, as they can capture both short-term and long-term patterns in the time-series data.

Model 2. Bi-Directional LSTM.

```

# save this as prabal_ghosh_6_bio . py

import numpy as np
import scipy.io as sio
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset, random_split
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# === LOAD & PREP DATA ===

# Load Ra files

ra1 = sio.loadmat('/home/prghosh/biomedical/Atrial_prabal_ghosh/Ra1.mat')['Ra1']
ra2 = sio.loadmat('/home/prghosh/biomedical/Atrial_prabal_ghosh/Ra2.mat')['Ra2']
ra3 = sio.loadmat('/home/prghosh/biomedical/Atrial_prabal_ghosh/Ra3.mat')['Ra3']
# Merge into one big 3D array      shape: (12, 15000, 75)
Xa_all = np.concatenate([ra1, ra2, ra3], axis=2)

# Extract only V1 V6 (indices 6 to 11)
Xa_v1_v6 = Xa_all[6:12, :, :] # shape: (6, 15000, 75)

# Rearrange to (75 patients, 15000 time steps, 6 leads)
X = np.transpose(Xa_v1_v6, (2, 1, 0)) # shape: (75, 15000, 6)

# === LABELS ===

# Load recurrence labels
indrecur = sio.loadmat('/home/prghosh/biomedical/Atrial_prabal_ghosh/indrecur.mat')['indrecur']
            .flatten() - 1
indnonrecur = sio.loadmat('/home/prghosh/biomedical/Atrial_prabal_ghosh/indnonrecur.mat')['
    indnonrecur'].flatten() - 1

# Create label vector: 0 = no recurrence, 1 = recurrence
y = np.zeros(75)
y[indrecur] = 1

# Convert data to PyTorch tensors

```

```

X_tensor = torch.tensor(X, dtype=torch.float32)
y_tensor = torch.tensor(y, dtype=torch.float32)

# Create a dataset and split into train and test sets
dataset = TensorDataset(X_tensor, y_tensor)
train_size = int(0.8 * len(dataset))
test_size = len(dataset) - train_size
train_dataset, test_dataset = random_split(dataset, [train_size, test_size])

train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=8, shuffle=False)

# === DEFINE BiLSTM MODEL ===
class BiLSTMModel(nn.Module):
    def __init__(self):
        super(BiLSTMModel, self).__init__()
        self.lstm1 = nn.LSTM(input_size=6, hidden_size=64, num_layers=1, batch_first=True,
                               bidirectional=True)
        self.dropout1 = nn.Dropout(0.2)
        self.lstm2 = nn.LSTM(input_size=128, hidden_size=64, num_layers=1, batch_first=True,
                               bidirectional=True)
        self.dropout2 = nn.Dropout(0.2)
        self.fc1 = nn.Linear(128, 64)
        self.dropout3 = nn.Dropout(0.5)
        self.fc2 = nn.Linear(64, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x, _ = self.lstm1(x)
        x = self.dropout1(x)
        x, _ = self.lstm2(x)
        x = self.dropout2(x)
        x = x[:, -1, :] # Take the last time step's output
        x = self.fc1(x)
        x = self.dropout3(x)
        x = self.fc2(x)
        x = self.sigmoid(x)
        return x

# Initialize the model, loss function, and optimizer
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = BiLSTMModel().to(device)
criterion = nn.BCELoss() # Binary Cross-Entropy Loss
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# === TRAIN THE MODEL ===
epochs = 20
for epoch in range(epochs):
    model.train()
    train_loss = 0.0
    for inputs, targets in train_loader:
        inputs, targets = inputs.to(device), targets.to(device).unsqueeze(1)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()

    train_loss += loss.item()

    print(f"Epoch {epoch + 1}/{epochs}, Loss: {train_loss / len(train_loader):.4f}")

# === EVALUATE THE MODEL ===
model.eval()

```

```

y_true = []
y_pred = []
with torch.no_grad():
    for inputs, targets in test_loader:
        inputs, targets = inputs.to(device), targets.to(device).unsqueeze(1)
        outputs = model(inputs)
        y_true.append(targets.cpu().numpy())
        y_pred.append(outputs.cpu().numpy())

y_true = np.concatenate(y_true, axis=0)
y_pred = np.concatenate(y_pred, axis=0)
y_pred = (y_pred > 0.5).astype(int)

# Calculate Accuracy and Classification Report
acc = accuracy_score(y_true, y_pred)
print(f"\nTest Accuracy: {acc * 100:.2f}%")
print("\nConfusion Matrix:\n", confusion_matrix(y_true, y_pred))
print("\nClassification Report:\n", classification_report(y_true, y_pred))

```

```

# save this run_shell_prabal . sh as a shell script

#!/bin/bash
#OAR -q production
#OAR -l host=1/gpu=1
#OAR -l walltime=00:30:00
#OAR -p gpu_count > 0
#OAR -O OAR_%jobid%.out
#OAR -E OAR_%jobid%.err
# display some information about attributed resources

echo "=== Host and GPU Info ==="
hostname
nvidia-smi
nvcc --version

echo "=== Loading environment ==="
module load conda
module load cuda/11.8
conda activate mri_2025_4

echo "=== Checking PyTorch GPU Availability ==="
python3 -c "import torch; print('CUDA available:', torch.cuda.is_available()); print('Device:',
    torch.cuda.get_device_name(0))"

cd /home/prghosh/biomedical

# Run your script
python prabal_ghosh_5_bio.py

echo "=== Done ==="
conda deactivate

```

```

*** result -***

```

```

prghosh@fsophia:~/biomedical$ chmod +x run_shell_prabal.sh
prghosh@fsophia:~/biomedical$ oarsub -S ./run_shell_prabal.sh
# Redirecting to the 'abaca' queue (new name for the 'production' queue)
# Setting queue to: p2
# Tie job resource request for GPU to resources with GPU
# Set walltime to default (3600 s).
OAR_JOB_ID=1410829
prghosh@fsophia:~/biomedical$ oarstat -u

```



```
Epoch 6/20, Loss: 0.7033
Epoch 7/20, Loss: 0.6808
Epoch 8/20, Loss: 0.6941
Epoch 9/20, Loss: 0.6820
Epoch 10/20, Loss: 0.6702
Epoch 11/20, Loss: 0.6959
Epoch 12/20, Loss: 0.6808
Epoch 13/20, Loss: 0.6741
Epoch 14/20, Loss: 0.6823
Epoch 15/20, Loss: 0.6861
Epoch 16/20, Loss: 0.6780
Epoch 17/20, Loss: 0.6954
Epoch 18/20, Loss: 0.6824
Epoch 19/20, Loss: 0.6737
Epoch 20/20, Loss: 0.6840
```

Test Accuracy: 53.33%

Confusion Matrix:

```
[[8 0]
 [7 0]]
```

Classification Report:

	precision	recall	f1-score	support
0.0	0.53	1.00	0.70	8
1.0	0.00	0.00	0.00	7
accuracy			0.53	15
macro avg	0.27	0.50	0.35	15
weighted avg	0.28	0.53	0.37	15

=== Done ===

In this classification, the bi-directional LSTM also works poorly.

1.6 6) ANN for Ventricular activity removal (applied to Rva with learning on Rva and Ra). Ra is supposed to be the ground truth. (student choice for the software, Tensorflow ?) à Use only V1 to V6 leads (Ra(7:12,:) and Rva(7:12,:))

```
# save this as prabal_ghosh_6_bio.py

import numpy as np
import scipy.io as sio
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset, random_split
from sklearn.metrics import mean_squared_error

# === LOAD & PREP DATA ===

# Load Ra files

ra1 = sio.loadmat('/home/prghosh/biomedical/Atrial_prabal_ghosh/Ra1.mat')['Ra1']
ra2 = sio.loadmat('/home/prghosh/biomedical/Atrial_prabal_ghosh/Ra2.mat')['Ra2']
ra3 = sio.loadmat('/home/prghosh/biomedical/Atrial_prabal_ghosh/Ra3.mat')['Ra3']

# Merge into one big 3D array      shape: (12, 15000, 75)
Xa_all = np.concatenate([ra1, ra2, ra3], axis=2)

# Extract only V 1 V6 (indices 6 to 11)
Xa_v1_v6 = Xa_all[6:12, :, :] # shape: (6, 15000, 75)

# Rearrange to (75 patients, 15000 time steps, 6 leads)
```

```

X = np.transpose(Xa_v1_v6, (2, 1, 0)) # shape: (75, 15000, 6)

# Load Rva files
rva1 = sio.loadmat('/home/prghosh/biomedical/Atrial_prabal_ghosh/Rva1.mat')['Rva1']
rva2 = sio.loadmat('/home/prghosh/biomedical/Atrial_prabal_ghosh/Rva2.mat')['Rva2']
rva3 = sio.loadmat('/home/prghosh/biomedical/Atrial_prabal_ghosh/Rva3.mat')['Rva3']

Rva_all = np.concatenate([rva1, rva2, rva3], axis=2)
Rva_v1_v6 = Rva_all[6:12, :, :] # Extract V 1 V6
Rva = np.transpose(Rva_v1_v6, (2, 1, 0)) # Rearrange to (75, 15000, 6)

# Convert data to PyTorch tensors
X_tensor = torch.tensor(X, dtype=torch.float32)
Rva_tensor = torch.tensor(Rva, dtype=torch.float32)

# split into train and test sets
dataset = TensorDataset(Rva_tensor, X_tensor)
train_size = int(0.8 * len(dataset))
test_size = len(dataset) - train_size
train_dataset, test_dataset = random_split(dataset, [train_size, test_size])

train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=4, shuffle=False)

# === BiLSTM MODEL ===
class BiLSTMModel(nn.Module):
    def __init__(self):
        super(BiLSTMModel, self).__init__()
        self.lstm1 = nn.LSTM(input_size=6, hidden_size=64, num_layers=1, batch_first=True,
                               bidirectional=True)
        self.dropout1 = nn.Dropout(0.2)
        self.lstm2 = nn.LSTM(input_size=128, hidden_size=64, num_layers=1, batch_first=True,
                               bidirectional=True)
        self.dropout2 = nn.Dropout(0.2)
        self.fc = nn.Linear(128, 6)

    def forward(self, x):
        x, _ = self.lstm1(x)
        x = self.dropout1(x)
        x, _ = self.lstm2(x)
        x = self.dropout2(x)
        x = self.fc(x)
        return x

# Initialize the model, loss function, and optimizer
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = BiLSTMModel().to(device)
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# === TRAIN THE MODEL ===
epochs = 10
for epoch in range(epochs):
    model.train()
    train_loss = 0.0
    for inputs, targets in train_loader:
        inputs, targets = inputs.to(device), targets.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()

    train_loss += loss.item()

```



```

    print(f"Epoch {epoch + 1}/{epochs}, Loss: {train_loss / len(train_loader):.4f}")

# === EVALUATE THE MODEL ===
model.eval()
y_true = []
y_pred = []
with torch.no_grad():
    for inputs, targets in test_loader:
        inputs, targets = inputs.to(device), targets.to(device)
        outputs = model(inputs)
        y_true.append(targets.cpu().numpy())
        y_pred.append(outputs.cpu().numpy())

y_true = np.concatenate(y_true, axis=0)
y_pred = np.concatenate(y_pred, axis=0)

# Calculate Mean Squared Error
mse = mean_squared_error(y_true.flatten(), y_pred.flatten())
print("Test MSE:", mse)

```

```

# save this run_shell_prabal.sh as a shell script

#!/bin/bash
#OAR -q production
#OAR -l host=1/gpu=1
#OAR -l walltime=00:30:00
#OAR -p gpu_count > 0
#OAR -O OAR_%jobid%.out
#OAR -E OAR_%jobid%.err
# display some information about attributed resources

echo "=== Host and GPU Info ==="
hostname
nvidia-smi
nvcc --version

echo "=== Loading environment ==="
module load conda
module load cuda/11.8
conda activate mri_2025_4

echo "=== Checking PyTorch GPU Availability ==="
python3 -c "import torch; print('CUDA available:', torch.cuda.is_available()); print('Device:',
    torch.cuda.get_device_name(0))"

cd /home/prghosh/biomedical

# Run your script
python prabal_ghosh_6_bio.py

echo "=== Done ==="
conda deactivate

```

result

```

prghosh@fsophia:~$ cd biomedical/
prghosh@fsophia:~/biomedical$ module load conda
prghosh@fsophia:~/biomedical$ module load cuda
prghosh@fsophia:~/biomedical$ vim run_shell_prabal.sh
prghosh@fsophia:~/biomedical$ chmod +x run_shell_prabal.sh
prghosh@fsophia:~/biomedical$ oarsub -S ./run_shell_prabal.sh

```

```
# Redirecting to the 'abaca' queue (new name for the 'production' queue)
# Setting queue to: p2
# Tie job resource request for GPU to resources with GPU
# Set walltime to default (3600 s).
OAR_JOB_ID=1410501
prghosh@fsophia:~/biomedical$ oarstat -u
Job id      Name      User      Submission Date      S Queue
-----
1410501      prghosh      2025-04-21 01:49:25 R p2
prghosh@fsophia:~/biomedical$ chmod +x run_shell_prabal.sh
prghosh@fsophia:~/biomedical$ oarstat -u
prghosh@fsophia:~/biomedical$ cat OAR_1410501.err
prghosh@fsophia:~/biomedical$ cat OAR_1410501.out
=== Host and GPU Info ===
musa-2.sophia.grid5000.fr
Mon Apr 21 01:49:29 2025
+-----+
| NVIDIA-SMI 535.183.06      Driver Version: 535.183.06      CUDA Version: 12.2      |
+-----+-----+-----+-----+-----+-----+
| GPU  Name      Persistence-M | Bus-Id      Disp.A | Volatile Uncorr. ECC |
| Fan  Temp      Perf      Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                               MIG M. |
+-----+-----+-----+-----+-----+-----+
| 0    NVIDIA H100 NVL      On | 00000000:82:00.0 Off |           0 |
| N/A   47C      P0              67W / 400W |      0MiB / 95830MiB |      0%      Default |
|                               Disabled |
+-----+-----+-----+-----+-----+-----+
| 1    NVIDIA H100 NVL      On | 00000000:E3:00.0 Off |           0 |
| N/A   44C      P0              64W / 400W |      0MiB / 95830MiB |      0%      Default |
|                               Disabled |
+-----+-----+-----+-----+-----+-----+
+-----+
| Processes:
| GPU  GI  CI      PID  Type  Process name      GPU Memory
|      ID  ID                               Usage
+-----+
| No running processes found
+-----+
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2021 NVIDIA Corporation
Built on Sun_Feb_14_21:12:58_PST_2021
Cuda compilation tools, release 11.2, V11.2.152
Build cuda_11.2.r11.2/compiler.29618528_0
=== Loading environment ===
=== Checking PyTorch GPU Availability ===
CUDA available: True
Device: NVIDIA H100 NVL
Epoch 1/10, Loss: 0.0071
Epoch 2/10, Loss: 0.0061
Epoch 3/10, Loss: 0.0059
Epoch 4/10, Loss: 0.0056
Epoch 5/10, Loss: 0.0054
Epoch 6/10, Loss: 0.0051
Epoch 7/10, Loss: 0.0047
Epoch 8/10, Loss: 0.0045
Epoch 9/10, Loss: 0.0040
Epoch 10/10, Loss: 0.0046
Test MSE: 0.0003438552957959473
=== Done ===
```

In this script, I have used a bidirectional LSTM (BiLSTM) model because the data is time-series in nature and spans a long sequence. The bidirectional LSTM is well-suited for capturing both past and future dependencies in such data.

I trained the model on the INRIA cluster because my local computer does not have a GPU, which is essential for efficiently training deep learning models like this one. The training process leverages the GPU to handle the computationally intensive operations.

The model achieved a Mean Squared Error (MSE) of 0.0003438552957959 on the test set. This very small MSE indicates that the model is performing exceptionally well in predicting the target value.